```tsx
/**
 * components/ml/FeedbackNudge.tsx
 *
 * Recommendation #2 — The feedback loop is built but not incentivised.
 *
 * When a work order is marked complete, this component surfaces a non-blocking
 * prompt: "Was the failure prediction accurate?" with a single thumbs up/down.
 * The operator can dismiss it — but the nudge means prediction feedback
 * actually gets submitted, making the retraining queue useful in practice.
 *
 * Usage — drop inside any work order completion flow:
 *
 *   <FeedbackNudge
 *     predictionId={prediction.id}
 *     predictionType={prediction.type}
 *     equipmentId={equipment.id}
 *     equipmentName={equipment.name}
 *     onDismiss={() => setShowNudge(false)}
 *   />
 */

import { useState } from 'react';
import { useMutation } from '@tanstack/react-query';
import { apiRequest } from '@/lib/queryClient';
import { Button } from '@/components/ui/button';
import { Badge } from '@/components/ui/badge';
import { Textarea } from '@/components/ui/textarea';
import { useToast } from '@/hooks/use-toast';
import { ThumbsUp, ThumbsDown, X, CheckCircle2, Brain } from 'lucide-react';
import { cn } from '@/lib/utils';

// ─── Types ────────────────────────────────────────────────────────────────────

interface FeedbackNudgeProps {
  predictionId: number;
  predictionType: string;
  equipmentId: string;
  equipmentName: string;
  onDismiss: () => void;
  className?: string;
}

type AccurateVote = true | false | null;
```

```
// ─── Component ──────────────────────────────────────────────────────────────

export function FeedbackNudge({
  predictionId,
  predictionType,
  equipmentId,
  equipmentName,
  onDismiss,
  className,
}: FeedbackNudgeProps) {
  const { toast } = useToast();
  const [vote, setVote] = useState<AccurateVote>(null);
  const [comments, setComments] = useState('');
  const [showComments, setShowComments] = useState(false);
  const [submitted, setSubmitted] = useState(false);

  const submitMutation = useMutation({
    mutationFn: async (payload: {
      predictionId: number;
      predictionType: string;
      equipmentId: string;
      feedbackType: string;
      isAccurate: boolean;
      comments: string | null;
    }) => {
      return apiRequest('POST', '/api/analytics/prediction-feedback', payload);
    },
    onSuccess: () => {
      setSubmitted(true);
      toast({
        title: 'Feedback recorded',
        description: 'Thank you — this helps improve prediction accuracy.',
      });
      // Auto-dismiss after 2 seconds
      setTimeout(onDismiss, 2000);
    },
    onError: () => {
      toast({
        title: 'Could not save feedback',
        description: 'Please try again.',
        variant: 'destructive',
      });
    },
  });

  const handleVote = (accurate: boolean) => {
    setVote(accurate);
```

```
  // For thumbs-down, show comment box to capture what was wrong
  if (!accurate) setShowComments(true);
};

const handleSubmit = () => {
  if (vote === null) return;
  submitMutation.mutate({
    predictionId,
    predictionType,
    equipmentId,
    feedbackType: 'rating',
    isAccurate: vote,
    comments: comments.trim() || null,
  });
};

// ── Submitted state ──────────────────────────────────────────────

if (submitted) {
  return (
    <div className={cn('flex items-center gap-2 p-3 rounded-lg bg-emerald-50 da
      <CheckCircle2 className="h-4 w-4 shrink-0" />
      Feedback recorded — helps improve future predictions
    </div>
  );
}

// ── Default state ────────────────────────────────────────────────

return (
  <div
    className={cn(
      'rounded-lg border border-border bg-muted/30 p-4 space-y-3',
      className,
    )}
    data-testid="feedback-nudge"
  >
    {/* Header */}
    <div className="flex items-start justify-between gap-2">
      <div className="flex items-center gap-2">
        <div className="p-1.5 rounded-md bg-blue-50 dark:bg-blue-950">
          <Brain className="h-4 w-4 text-blue-600 dark:text-blue-400" />
        </div>
        <div>
          <p className="text-sm font-medium">Was the prediction accurate?</p>
          <p className="text-xs text-muted-foreground mt-0.5">
            <span className="font-medium">{equipmentName}</span> · {predictionT
```

```
      </p>
    </div>
  </div>
  <button
    onClick={onDismiss}
    className="text-muted-foreground hover:text-foreground transition-color
    aria-label="Dismiss"
    data-testid="button-dismiss-nudge"
  >
    <X className="h-4 w-4" />
  </button>
</div>

{/* Vote buttons */}
<div className="flex gap-2">
  <Button
    size="sm"
    variant={vote === true ? 'default' : 'outline'}
    onClick={() => handleVote(true)}
    className={cn(
      'gap-1.5 flex-1',
      vote === true && 'bg-emerald-600 hover:bg-emerald-700 border-emerald-
    )}
    data-testid="button-vote-accurate"
  >
    <ThumbsUp className="h-3.5 w-3.5" />
    Yes, accurate
  </Button>
  <Button
    size="sm"
    variant={vote === false ? 'default' : 'outline'}
    onClick={() => handleVote(false)}
    className={cn(
      'gap-1.5 flex-1',
      vote === false && 'bg-red-600 hover:bg-red-700 border-red-600',
    )}
    data-testid="button-vote-inaccurate"
  >
    <ThumbsDown className="h-3.5 w-3.5" />
    Not accurate
  </Button>
</div>

{/* Optional comment — auto-shown for thumbs-down */}
{(showComments || vote !== null) && (
  <div className="space-y-2">
    {!showComments && (
```

```
            <button
              onClick={() => setShowComments(true)}
              className="text-xs text-muted-foreground underline underline-offset
            >
              Add a comment (optional)
            </button>
          )}
          {showComments && (
            <Textarea
              placeholder={
                vote === false
                  ? 'What was wrong with the prediction? (e.g. failure happened s
                  : 'Any additional notes?'
              }
              value={comments}
              onChange={e => setComments(e.target.value)}
              rows={2}
              className="text-sm resize-none"
              data-testid="textarea-feedback-comments"
            />
          )}
        </div>
      )}

      {/* Submit — only shown after a vote */}
      {vote !== null && (
        <div className="flex items-center justify-between">
          <Badge variant="outline" className="text-xs">
            Your feedback helps retrain the AI model
          </Badge>
          <Button
            size="sm"
            onClick={handleSubmit}
            disabled={submitMutation.isPending}
            data-testid="button-submit-feedback"
          >
            {submitMutation.isPending ? 'Saving…' : 'Submit'}
          </Button>
        </div>
      )}
    </div>
  );
}
```